



UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
CARRERA DE ING. INFORMÁTICA



HERRAMIENTAS DE VISIÓN ARTIFICIAL

Estudiantes:

1. Muñoz Camila Andrea
2. Vargas Morales Gerald Emanuel
3. Alanes Torrez Salvador David
4. Tola Mariño Angel
5. Gonzales Tincuta Josue Arturo

Docente: Lic. Carmen Rosa García Pérez

Asignatura: Inteligencia Artificial II

Fecha: 03/06/2026

1. Introducción y Entorno de Trabajo

Para el desarrollo de las prácticas de visión artificial se utilizaron herramientas de programación y procesamiento de imágenes ampliamente empleadas en el ámbito académico e industrial. Estas herramientas permiten adquirir, manipular, analizar e interpretar imágenes digitales mediante algoritmos computacionales.

Python

Python es un lenguaje de programación de alto nivel, interpretado y multiparadigma, ampliamente utilizado en inteligencia artificial, análisis de datos, aprendizaje automático y visión artificial. Su sintaxis sencilla y la gran cantidad de bibliotecas disponibles facilitan el desarrollo de aplicaciones científicas y tecnológicas.

OpenCV (Open Source Computer Vision Library)

Es una biblioteca de código abierto especializada en visión artificial y procesamiento digital de imágenes. Proporciona funciones para captura de imágenes y video, filtrado, segmentación, detección de objetos, reconocimiento de patrones y seguimiento de movimiento.

NumPy

Es una biblioteca fundamental para el cálculo numérico en Python. Permite trabajar con arreglos multidimensionales y realizar operaciones matemáticas de manera eficiente. En visión artificial, las imágenes son representadas internamente como matrices NumPy.

Matplotlib

Matplotlib es una biblioteca utilizada para la visualización de datos y gráficos. En visión artificial permite mostrar imágenes, histogramas y resultados de procesamiento de manera sencilla.

Google Colab

Google Colaboratory (Google Colab) es una plataforma basada en la nube que permite ejecutar cuadernos de Python directamente desde el navegador. Proporciona acceso gratuito a recursos computacionales, almacenamiento temporal y soporte para bibliotecas de inteligencia artificial y visión artificial sin necesidad de configurar un entorno local.

Importación de Librerías

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from scipy.stats import norm
import math
```

Utilidades de Visualización

Con el objetivo de mostrar los resultados obtenidos durante el procesamiento de imágenes se implementaron funciones auxiliares de visualización.

Código:

```
def mostrar_imagen(titulo, imagen, cmap='gray'):
    plt.figure(figsize=(6, 5))
    plt.imshow(imagen, cmap=cmap)
    plt.title(titulo)
    plt.axis('off')
    plt.tight_layout()
    plt.show()
```

La función `mostrar_imagen()` recibe una imagen y un título como parámetros. Internamente utiliza la biblioteca Matplotlib para crear una ventana gráfica donde se visualiza la imagen procesada. La instrucción `plt.imshow()` muestra la imagen, mientras que `plt.title()` agrega un título descriptivo. La línea `plt.axis('off')` elimina los ejes de coordenadas para facilitar la observación del contenido visual. Finalmente, `plt.show()` presenta el resultado en pantalla.

Código:

```
def mostrar_varias(imagenes_titulos, cols=3, cmap='gray'):
    n = len(imagenes_titulos)
    rows = math.ceil(n / cols)
    fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 4 * rows))
    axes = np.array(axes).flatten()
    for i, (img, titulo) in enumerate(imagenes_titulos):
```

```

cm = 'gray' if len(img.shape) == 2 else None
axes[i].imshow(img, cmap=cm)
axes[i].set_title(titulo)
axes[i].axis('off')
for j in range(i + 1, len(axes)):
    axes[j].axis('off')
plt.tight_layout()
plt.show()

```

La función `mostrar_varias()` permite visualizar simultáneamente múltiples imágenes en una misma figura. Primero calcula la cantidad de filas y columnas necesarias para organizar las imágenes mediante la instrucción `math.ceil()`. Posteriormente utiliza `plt.subplots()` para generar una cuadrícula de gráficos donde cada posición almacena una imagen y su respectivo título. Esta función se emplea principalmente para comparar resultados obtenidos durante diferentes etapas del procesamiento, como ocurre en la demostración de cuantización, donde se muestran imágenes con distintos niveles de intensidad.

2. Formación y representación de imágenes

2.1 Imagen Digital y Píxel

La función `crear_imagen_ejemplo()` crea una imagen digital artificial representada mediante una matriz de 8 filas por 8 columnas. Cada elemento de la matriz corresponde a un píxel y almacena un valor de intensidad comprendido entre 0 y 255. El valor 0 representa negro, el valor 255 representa blanco y los valores intermedios representan distintos niveles de gris.

La instrucción **`imagen = np.array([...], dtype=np.uint8)`** crea la matriz utilizando el tipo de dato `uint8` (entero sin signo de 8 bits), que es el formato estándar para almacenar imágenes en escala de grises de 256 niveles de intensidad.

Los valores de la matriz fueron definidos manualmente para generar una transición gradual desde intensidades bajas hasta intensidades altas. Por ejemplo:

- `imagen[0,0] = 0` corresponde a un píxel negro, mientras que:
- `imagen[7,7] = 255` corresponde a un píxel blanco.

Posteriormente, las línea `print(f"Imagen digital creada:`

`{imagen.shape[0]}×{imagen.shape[1]} pixeles")` muestran el tamaño de la imagen.

La propiedad `shape` devuelve las dimensiones de la matriz, donde **`shape[0]`** indica el número de filas y **`shape[1]`** el número de columnas.

La línea `print(f"I(x,y) ∈ R(M×N) donde M={imagen.shape[0]},`

`N={imagen.shape[1]}")` muestra la representación matemática de la imagen digital como una matriz $M \times N$, relacionando directamente la teoría con la estructura utilizada en el programa.

Finalmente **`return imagen`** devuelve la matriz creada para que pueda ser utilizada posteriormente por otras funciones del sistema, como las operaciones de vecindad, cálculo de distancias o procesamiento de imágenes.

Código:

```
def crear_imagen_ejemplo(M=8, N=8):
```

```
    imagen = np.array([
```

```
        [0, 10, 20, 30, 40, 50, 60, 70 ],
```

```
        [10, 30, 60, 90, 120, 150, 180, 210],
```

```
        [20, 60, 100, 140, 180, 200, 220, 240],
```

```
        [30, 90, 140, 180, 200, 210, 220, 240],
```

```
        [40, 120, 180, 200, 210, 215, 225, 245],
```

```
        [50, 150, 200, 210, 215, 220, 230, 250],
```

```
        [60, 180, 220, 220, 225, 230, 240, 252],
```

```
        [70, 210, 240, 240, 245, 250, 252, 255],
```

```
    ], dtype=np.uint8)
```

```
    print(f"Imagen digital creada: {imagen.shape[0]}×{imagen.shape[1]} pixeles")
```

```
    print(f"I(x,y) ∈ R(M×N) donde M={imagen.shape[0]}, N={imagen.shape[1]}")
```

```
    return imagen
```

Vecindad de un píxel

Vecindad N4.

La función `vecindad_4()` implementa el concepto de vecindad de un píxel utilizando la conectividad de cuatro vecinos (N4). Su objetivo es identificar los píxeles adyacentes a

una posición determinada dentro de la imagen. Para ello recibe como parámetros la imagen y las coordenadas (x,y) del píxel que se desea analizar.

Inicialmente se obtienen las dimensiones de la imagen mediante: `filas, cols = imagen.shape`

Posteriormente se verifica si existen vecinos válidos alrededor del píxel seleccionado.

Por ejemplo:

```
if x > 0:
```

```
    vecinos['arriba'] = imagen[x-1, y]
```

comprueba que el píxel no se encuentre en el borde superior de la imagen antes de acceder al vecino ubicado arriba. El mismo procedimiento se aplica para los vecinos inferior, izquierdo y derecho.

El resultado se almacena en un diccionario llamado **vecinos**, donde cada clave indica la posición relativa del vecino y cada valor corresponde a la intensidad del píxel encontrado. Esta función permite representar computacionalmente la vecindad N4 estudiada en teoría, formada únicamente por los vecinos horizontal y verticalmente adyacentes.

Código:

```
def vecindad_4(imagen, x, y):
    filas, cols = imagen.shape
    vecinos = {}
    if x > 0:      vecinos['arriba']  = imagen[x-1, y]
    if x < filas - 1: vecinos['abajo']  = imagen[x+1, y]
    if y > 0:      vecinos['izquierda'] = imagen[x, y-1]
    if y < cols - 1: vecinos['derecha'] = imagen[x, y+1]
    return vecinos
```

Vecindad N8.

La función `vecindad_8()` amplía el concepto anterior incorporando también los vecinos diagonales. En lugar de volver a calcular los vecinos horizontales y verticales, reutiliza directamente la función N4 mediante la instrucción: `vecinos = vecindad_4(imagen, x, y)`. De esta manera se obtienen primero los cuatro vecinos básicos y luego se agregan las diagonales, por ejemplo:

if $x > 0$ and $y > 0$:

```
vecinos['sup-izq'] = imagen[x-1, y-1]
```

verifica que exista una posición válida en la diagonal superior izquierda antes de acceder a ella. El mismo procedimiento se realiza para las diagonales superior derecha, inferior izquierda e inferior derecha.

Al finalizar, se contiene hasta ocho vecinos alrededor del píxel analizado. Este bloque implementa directamente el concepto teórico de vecindad N8, donde se consideran tanto los vecinos horizontales y verticales como los diagonales. La función es necesaria para estudiar relaciones espaciales más completas dentro de una imagen y constituye la base para conceptos posteriores como conectividad, segmentación y análisis de regiones.

Código:

```
def vecindad_8(imagen, x, y):
```

```
    filas, cols = imagen.shape
```

```
    vecinos = vecindad_4(imagen, x, y)
```

```
    if  $x > 0$  and  $y > 0$ :        vecinos['sup-izq'] = imagen[x-1, y-1]
```

```
    if  $x > 0$  and  $y < cols-1$ :    vecinos['sup-der'] = imagen[x-1, y+1]
```

```
    if  $x < filas-1$  and  $y > 0$ :    vecinos['inf-izq'] = imagen[x+1, y-1]
```

```
    if  $x < filas-1$  and  $y < cols-1$ : vecinos['inf-der'] = imagen[x+1, y+1]
```

```
    return vecinos
```

Distancia entre píxeles

1. Distancia Euclidiana.

Esta función implementa la distancia euclidiana entre dos píxeles. Los parámetros p y q representan las coordenadas de dos posiciones dentro de la imagen. En las primeras líneas se extraen las coordenadas de cada punto: $x, y = p \mid u, v = q$ donde $((x,y))$ corresponde al primer píxel y $((u,v))$ al segundo.

La distancia se calcula utilizando la fórmula geométrica clásica:

$$D_e(p, q) = \sqrt{(x - u)^2 + (y - v)^2}$$

Esta ecuación corresponde al teorema de Pitágoras aplicado al plano cartesiano. En el código, la función `np.sqrt()` calcula la raíz cuadrada mientras que los términos elevados

al cuadrado representan las diferencias entre coordenadas horizontales y verticales. El resultado obtenido corresponde a la distancia real en línea recta entre ambos píxeles.

Código:

```
def distancia_euclidiana(p, q):  
    x, y = p  
    u, v = q  
    return np.sqrt((x - u)**2 + (y - v)**2)
```

2. Distancia Manhattan.

Esta función implementa la distancia Manhattan, también conocida como distancia D4. Al igual que en la función anterior, primero se obtienen las coordenadas de los dos píxeles.

La distancia se calcula mediante la fórmula:

$$D_4(p, q) = |x - u| + |y - v|$$

La función **abs()** calcula el valor absoluto de cada diferencia para evitar resultados negativos. Posteriormente ambas diferencias se suman. Esta métrica considera únicamente desplazamientos horizontales y verticales, siguiendo la misma lógica utilizada en la conectividad N4.

Código:

```
def distancia_manhattan(p, q):  
    x, y = p  
    u, v = q  
    return abs(x - u) + abs(y - v)
```

3. Distancia Chessboard.

Esta función implementa la distancia Chessboard o distancia D8. Después de obtener las coordenadas de ambos píxeles, se calcula la diferencia absoluta entre filas y columnas.

La distancia se determina mediante la expresión:

$$D_8(p, q) = \max(|x - u|, |y - v|)$$

La función `max()` selecciona la mayor de las dos diferencias. Esta métrica considera desplazamientos horizontales, verticales y diagonales, de forma similar a los movimientos de una reina en un tablero de ajedrez.

Código:

```
def distancia_chessboard(p, q):  
    x, y = p  
    u, v = q  
    return max(abs(x - u), abs(y - v))
```

Demostración de distancias entre píxeles

La función `demo_distancias()` tiene como objetivo mostrar de forma práctica el funcionamiento de las tres métricas de distancia implementadas anteriormente: Euclidiana, Manhattan y Chessboard.

Dentro de la función se definen dos píxeles de prueba: $p = (0,0)$ y $q = (3,4)$. Estas coordenadas representan posiciones dentro de una imagen digital. A continuación, la función llama a cada una de las métricas implementadas y muestra en pantalla tanto la fórmula utilizada como el resultado obtenido.

Código:

```
def demo_distancias():  
    p = (0, 0)  
    q = (3, 4)  
    print("Distancias entre pixeles ===")  
    print(f"Pixel p={p}, pixel q={q}")  
    print(f" Euclidiana De = sqrt({q[0]}^2+{q[1]}^2) = {distancia_euclidiana(p,q):.4f}")  
    print(f" Manhattan D4 = |{q[0]}|+|{q[1]}| = {distancia_manhattan(p,q)}")  
    print(f" Chessboard D8 = max({q[0]},{q[1]}) = {distancia_chessboard(p,q)}")
```

2.2 Resolución

Resolución Espacial

La función `crear_imagen_ejemplo()` permite representar el concepto de resolución espacial mediante las dimensiones de la matriz que almacena la imagen digital. La resolución espacial está determinada por la cantidad de píxeles utilizados para

representar una imagen, expresada como el número de filas M y columnas N. En el código, la instrucción:

- `imagen.shape[0]`
- `imagen.shape[1]`

obtiene respectivamente la cantidad de filas y columnas de la imagen, mostrando su tamaño en píxeles. La matriz creada posee una resolución de 8×8 píxeles, lo que significa que la imagen está formada por 64 muestras espaciales.

Resolución Radiométrica

La función **calcular_niveles_intensidad()** implementa el concepto de resolución radiométrica, que corresponde a la cantidad de niveles de intensidad que puede representar cada píxel. La función recibe como parámetro el número de bits k utilizados para almacenar la intensidad y aplica la ecuación:

$$L=2^k$$

donde L representa el número total de niveles de intensidad posibles. Por ejemplo, para una imagen de 8 bits se obtienen 256 niveles de gris. El objetivo de esta función es calcular y mostrar cuántos valores distintos puede almacenar un píxel según la cantidad de bits utilizada.

Código:

```
def calcular_niveles_intensidad(k_bits):  
    L = 2 ** k_bits  
    print(f"=== Niveles de Intensidad ===")  
    print(f" L = 2^k = 2^{k_bits} = {L} niveles")  
    return L
```

2.3 Profundidad de Bits

Bits por píxel

La profundidad de bits corresponde a la cantidad de bits utilizados para almacenar la información de cada píxel. En el código, este concepto está representado por el parámetro `k_bits` de la función **calcular_niveles_intensidad()**. Al modificar dicho valor es posible observar cómo cambia la capacidad de representación de información en la imagen.

Niveles de gris

La misma función permite calcular la cantidad de niveles de gris disponibles para una determinada profundidad de bits utilizando la ecuación:

$$L=2^k$$

Por ejemplo:

- 8 bits → 256 niveles de gris.
- 10 bits → 1024 niveles de gris.
- 12 bits → 4096 niveles de gris.

De esta forma, el código relaciona directamente la cantidad de bits utilizados con el número de intensidades que puede representar cada píxel.

Profundidad de color RGB

La función `calcular_colores_rgb()` calcula la cantidad total de colores que pueden representarse en una imagen RGB. Como una imagen RGB posee tres canales (Rojo, Verde y Azul), cada uno con una determinada cantidad de bits, el número total de colores se obtiene mediante:

$$Colores = 2^{3k}$$

donde k representa los bits utilizados por cada canal. Para el caso habitual de 8 bits por canal se obtienen 16 777 216 colores posibles.

Código:

```
def calcular_colores_rgb(bits_por_canal=8):  
    total = 2 ** (3 * bits_por_canal)  
    print(f"===Colores posibles en RGB ===")  
    print(f" 2^(3×{bits_por_canal}) = 2^{3*bits_por_canal} = {total:,} colores")  
    return total
```

Las funciones `calcular_niveles_intensidad()` y `calcular_colores_rgb()` muestran que al aumentar la cantidad de bits utilizados también aumenta la cantidad de información que debe almacenarse para cada píxel. Por esta razón, una mayor profundidad de bits permite representar más niveles de intensidad o más colores, pero requiere una mayor cantidad de memoria y espacio de almacenamiento para la imagen.

2.4 Muestreo y Cuantización

Cuantización

La función `demostrar_cuantizacion()` implementa el proceso de cuantización reduciendo la cantidad de niveles de intensidad disponibles en una imagen. Para ello utiliza diferentes cantidades de niveles (256, 64, 16 y 4) y fuerza a que cada píxel tome únicamente valores pertenecientes a dichos niveles. La operación:

```
img_cuantizada = (imagen_gris // factor) * factor
```

agrupa múltiples intensidades originales dentro de un mismo nivel cuantizado. Esto permite observar visualmente el efecto de la reducción de niveles de gris. A medida que disminuye la cantidad de niveles aparecen pérdidas de detalle, ruido de cuantización y el efecto conocido como *banding*.

Código:

```
def demostrar_cuantizacion(imagen_gris, niveles_lista=[256, 64, 16, 4]):
    resultados = []
    for niveles in niveles_lista:
        factor = 256 // niveles
        img_cuantizada = (imagen_gris // factor) * factor
        resultados.append((img_cuantizada, f"L={niveles} niveles"))
    mostrar_varias(resultados, cols=len(niveles_lista))
    print("Con pocos niveles aparece ruido de cuantizacion y banding")
```

Muestreo

La función `crear_imagen_ejemplo()` representa el resultado del proceso de muestreo. El muestreo consiste en discretizar espacialmente una imagen continua, transformándola en una matriz formada por píxeles. En el código, la imagen se almacena como una matriz de dimensiones $M \times N$, donde cada posición corresponde a una muestra tomada de la escena original. La instrucción `np.array()` crea dicha estructura matricial y cada elemento de la matriz representa un píxel ubicado en una coordenada específica (x,y). De esta forma, la función permite trabajar con una imagen digital compuesta por un número finito de muestras espaciales.

Formación de la Imagen Digital

La formación de la imagen digital se observa en la función `crear_imagen_ejemplo()`, donde una imagen es representada mediante una matriz de píxeles almacenados digitalmente. Cada elemento de la matriz contiene un valor de intensidad comprendido entre 0 y 255 gracias al tipo de dato `uint8`, que utiliza 8 bits por píxel. La imagen generada constituye el resultado de dos procesos fundamentales: el muestreo, que determina la posición de cada píxel dentro de la matriz, y la cuantización, que asigna a cada píxel un valor discreto de intensidad. Como resultado, la imagen queda representada computacionalmente como una función discreta:

$$f(x, y) \in \mathbb{R}^{M \times N}$$

donde M corresponde al número de filas y N al número de columnas de la imagen digital.

2.5 Espacios de color

RGB

Esta función trabaja con una imagen en formato RGB, donde cada píxel está compuesto por tres componentes de color: rojo (R), verde (G) y azul (B).

Matemáticamente, un píxel puede representarse como:

$$I(x, y) = (R, G, B)$$

El objetivo de la función es separar estos tres canales para poder analizarlos individualmente. La instrucción `imagen_rgb[:, :, 0]` obtiene todos los valores correspondientes al canal rojo, mientras que `imagen_rgb[:, :, 1]` y `imagen_rgb[:, :, 2]` obtienen los canales verde y azul respectivamente. Finalmente, la función devuelve los tres canales por separado. Esta separación es necesaria porque posteriormente pueden realizarse operaciones independientes sobre cada componente de color.

Código:

```
def separar_canales_rgb(imagen_rgb):  
    R = imagen_rgb[:, :, 0]  
    G = imagen_rgb[:, :, 1]
```

```
B = imagen_rgb[:, :, 2]
print("Espacio RGB — canales separados R, G, B")
return R, G, B
```

HSV

Esta función convierte una imagen almacenada en formato BGR al espacio de color HSV utilizando la función `cv2.cvtColor()`. El resultado de la conversión se almacena en la variable `hsv`.

Después el código extrae cada componente del espacio HSV. El canal H representa el tono, el canal S representa la saturación y el canal V representa el brillo. Esto se realiza mediante indexación matricial:

```
H = hsv[:, :, 0]
S = hsv[:, :, 1]
V = hsv[:, :, 2]
```

La función devuelve tanto los canales separados como la imagen completa en formato HSV. El propósito de esta implementación es disponer de cada componente por separado para poder analizarlas o utilizarlas en operaciones posteriores dentro del procesamiento de imágenes.

Código:

```
def convertir_hsv(imagen_bgr):
    hsv = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2HSV)
    H = hsv[:, :, 0]
    S = hsv[:, :, 1]
    V = hsv[:, :, 2]
    print("Espacio HSV — H(tono), S(saturacion), V(brillo) separados")
    return H, S, V, hsv
```

Escala de grises

Esta función convierte una imagen a escala de grises. Primero se extraen los canales azul, verde y rojo de la imagen original y se convierten al tipo `float64` para realizar operaciones matemáticas con mayor precisión.

La conversión se realiza mediante la ecuación donde (Y) representa la intensidad final del píxel en escala de grises.

$$Y = 0.299R + 0.587G + 0.114B$$

Una vez calculada la intensidad, la función utiliza `np.clip(Y, 0, 255)` para asegurar que los valores permanezcan dentro del rango permitido para una imagen de 8 bits.

Finalmente, `np.uint8()` convierte los resultados al formato entero utilizado para almacenar imágenes digitales.

Código:

```
def convertir_escala_grises(imagen_color):  
    B = imagen_color[:, :, 0].astype(np.float64)  
    G = imagen_color[:, :, 1].astype(np.float64)  
    R = imagen_color[:, :, 2].astype(np.float64)  
    # Suma ponderada  
    Y = 0.299 * R + 0.587 * G + 0.114 * B  
    return np.uint8(np.clip(Y, 0, 255))
```

YUV

Esta función convierte una imagen desde el formato BGR al espacio de color YUV utilizando la función `cv2.cvtColor()`. El resultado de la conversión se almacena en la variable `yuv`.

Después de la conversión, se extraen los tres componentes del nuevo espacio de color. El canal Y contiene la información de luminancia, mientras que los canales U y V contienen la información de crominancia. La extracción se realiza mediante:

```
Y = yuv[:, :, 0]  
U = yuv[:, :, 1]  
V = yuv[:, :, 2]
```

La función devuelve los tres canales por separado junto con la imagen completa en formato YUV. El propósito de esta implementación es disponer de cada componente de forma independiente para su análisis o procesamiento posterior dentro del sistema de visión artificial.

Código:

```
def convertir_yuv(imagen_bgr):
    yuv = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2YUV)
    Y = yuv[:, :, 0]
    U = yuv[:, :, 1]
    V = yuv[:, :, 2]
    print("Espacio YUV — Y(luminancia), U,V(crominancia) separados")
    return Y, U, V, yuv
```

3. Adquisición de Imágenes

La adquisición de imágenes es el primer eslabón del pipeline de visión artificial. Consiste en capturar información visual del entorno y convertirla en datos digitales que una computadora pueda procesar. La calidad con la que se realiza esta etapa condiciona directamente la eficacia de todos los pasos posteriores: un sistema no puede extraer información que nunca estuvo bien capturada

3.1 Cámaras CCD y CMOS

Las cámaras CCD y CMOS utilizan sensores fotosensibles para transformar luz en señales eléctricas.

```
def convertir_hsv(imagen_bgr):
    hsv = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2HSV)
    H = hsv[:, :, 0]
    S = hsv[:, :, 1]
    V = hsv[:, :, 2]
    print("Espacio HSV — H(tono), S(saturacion), V(brillo) separados")
    return H, S, V, hsv
```

3.2 Sensores de Imagen

Un sensor de imagen es una matriz de píxeles fotosensibles. Cada píxel almacena intensidad luminosa.

El proceso de captura sigue tres etapas: primero, la luz incidente genera pares electrón-hueco en el material semiconductor mediante el efecto fotoeléctrico; segundo,

la carga acumulada es leída y amplificada; y tercero, la señal es enviada al convertidor analógico-digital. Los parámetros que caracterizan a un sensor son:

- Resolución espacial: número total de píxeles del arreglo. Determina el nivel de detalle que puede capturarse.
- Tamaño del píxel: a mayor tamaño, mayor capacidad de captura de luz y menor ruido en condiciones de baja iluminación.
- Rango dinámico: cociente entre el valor máximo y mínimo de intensidad que el sensor puede registrar simultáneamente.
- Velocidad de lectura: tasa máxima de cuadros por segundo (fps) que el sensor puede producir.

```
def crear_imagen_ejemplo(M=8, N=8):
    imagen = np.array([
        [0, 10, 20, 30, 40, 50, 60, 70 ],
        [10, 30, 60, 90, 120, 150, 180, 210],
        [20, 60, 100, 140, 180, 200, 220, 240],
        [30, 90, 140, 180, 200, 210, 220, 240],
        [40, 120, 180, 200, 210, 215, 225, 245],
        [50, 150, 200, 210, 215, 220, 230, 250],
        [60, 180, 220, 220, 225, 230, 240, 252],
        [70, 210, 240, 240, 245, 250, 252, 255],
    ], dtype=np.uint8)
    print(f"Imagen digital creada: {imagen.shape[0]}x{imagen.shape[1]} pixeles")
    print(f" $I(x,y) \in \mathbb{R}^{(M \times N)}$  donde  $M={imagen.shape[0]}$ ,  $N={imagen.shape[1]}$ ")
    return imagen
```

3.3 Conversión Analógico-Digital

La señal analógica producida por el sensor debe convertirse a valores digitales discretos.

Niveles disponibles:

$$L = 2^b \text{ Para } L = 2^8 = L = 256 \text{ niveles (0 a 255)}$$

Codigo:

```
def calcular_niveles_intensidad(k_bits):
    L = 2 ** k_bits
    print(f"=== Niveles de Intensidad ===")
    print(f"    L = 2^k = 2^{k_bits} = {L} niveles")
    return L

def demostrar_cuantizacion(imagen_gris, niveles_lista=[256, 64, 16, 4]):
    resultados = []
    for niveles in niveles_lista:
        factor = 256 // niveles
        img_cuantizada = (imagen_gris // factor) * factor
        resultados.append((img_cuantizada, f"L={niveles} niveles"))
    mostrar_varias(resultados, cols=len(niveles_lista))
    print("Con pocos niveles aparece ruido de cuantizacion y banding")
```

3.4 Ruido en Imágenes

El ruido es una variación aleatoria e indeseable en los valores de intensidad de los píxeles, introducida durante la captura. Degrada la información visual y puede interferir con algoritmos de segmentación o detección de bordes, que son especialmente sensibles a variaciones locales de intensidad. Las fuentes de ruido más relevantes en sistemas de adquisición digital son:

- Ruido térmico: generado por el movimiento aleatorio de electrones en el sensor debido a la temperatura. Se reduce refrigerando el sensor, práctica común en astronomía y microscopía.
- Ruido de disparo (shot noise): originado en la naturaleza cuántica de la luz. Los fotones llegan al sensor de forma aleatoria siguiendo una distribución de Poisson, produciendo fluctuaciones inevitables.
- Ruido de lectura: introducido en la etapa de conversión de carga a voltaje. Su magnitud depende de la velocidad de lectura del sensor.
- Ruido de cuantización: resultado de aproximar valores continuos a niveles discretos durante la conversión ADC.

El modelo matemático estándar representa la imagen capturada como la suma de la imagen ideal más una perturbación de ruido:

$$I_{ruidosa}(x,y) = I_{original}(x,y) + n(x,y)$$

$n(x,y)$ es la función de ruido

Codigo:

```
def modelo_ruido_gaussiano(imagen, sigma=15):
    ruido = np.random.normal(0, sigma, imagen.shape)
    imagen_ruidosa = np.clip(imagen.astype(np.float64) + ruido, 0, 255)
    print(f"=== Modelo de Ruido ===")
    print(f" I_ruidosa = I_original + n(x,y)   [σ={sigma}]")
    return np.uint8(imagen_ruidosa)

def modelo_ruido_sal_pimienta(imagen, densidad=0.05):
    img_ruidosa = imagen.copy()
    n_pixeles = int(densidad * imagen.size)
    coords = [np.random.randint(0, d, n_pixeles) for d in imagen.shape]
    img_ruidosa[coords[0], coords[1]] = 255
    coords = [np.random.randint(0, d, n_pixeles) for d in imagen.shape]
    img_ruidosa[coords[0], coords[1]] = 0
    return img_ruidosa
```

3.5 Iluminación, Calibración y Perspectiva

La iluminación, la calibración de la cámara y los factores geométricos como la distorsión y la perspectiva son elementos estrechamente relacionados que afectan la fidelidad con la que la imagen representa la escena real. Su comprensión es fundamental para diseñar sistemas de visión confiables.

La iluminación es el factor más determinante en la calidad visual de la imagen. Una iluminación inadecuada genera sombras, reflejos especulares o bajo contraste que dificultan el análisis posterior. En sistemas industriales se utilizan esquemas controlados como la iluminación difusa frontal, la iluminación rasante para revelar texturas, o la iluminación trasera para obtener siluetas nítidas. Para sistemas que operan en exteriores, se deben considerar los cambios de iluminación por condiciones climáticas, ciclos día/noche y sombras en movimiento.

$$x_{corr} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

con $r^2 = x^2 + y^2$, k_i = coeficientes de distorsión

Codigo:

```
def correccion_distorsion_radial(imagen, k1=0.1, k2=0.01, k3=0.001):
    h, w = imagen.shape[:2]
    cx, cy = w / 2.0, h / 2.0

    map_x = np.zeros((h, w), np.float32)
    map_y = np.zeros((h, w), np.float32)

    for v in range(h):
        for u in range(w):
            x = (u - cx) / cx
            y = (v - cy) / cy
            r2 = x**2 + y**2
            factor = 1 + k1 * r2 + k2 * r2**2 + k3 * r2**3
            map_x[v, u] = cx + x * factor * cx
            map_y[v, u] = cy + y * factor * cy

    corregida = cv2.remap(imagen, map_x, map_y, cv2.INTER_LINEAR)
    print("Correccion de distorsion radial aplicada")
    return corregida
```

$$(u, v) = \left(f \cdot \frac{x}{z}, f \cdot \frac{y}{z}\right)$$

f = distancia focal, Z = profundidad del punto 3D

```
def proyeccion_perspectiva_pinhole(X, Y, Z, f=500):

    if Z == 0:
        raise ValueError("Z no puede ser 0 (division por cero)")
    u = f * X / Z
    v = f * Y / Z
    print(f"=== Proyeccion Pinhole ===")
    print(f" Punto 3D ({X}, {Y}, {Z}) → Pixel ({u:.2f}, {v:.2f}) [f={f}]")
    return u, v
```

4. Procesamiento Digital de Imágenes

El procesamiento digital de imágenes aplica transformaciones matemáticas sobre imágenes digitales para mejorar su calidad o preparar análisis posteriores.

4.1 Operaciones Puntuales

Las operaciones puntuales modifican cada píxel de manera independiente.

Negativo digital: invierte los niveles de intensidad, convirtiendo zonas oscuras en claras y viceversa. Es útil en imágenes médicas donde estructuras de interés (como masas en mamografías) son más visibles sobre fondo blanco.

$$I'(x, y) = (L - 1) - I(x, y)$$

Para 8 bits: $I'(x, y) = 255 - I(x, y)$

```
def negativo_digital(imagen):  
    return np.uint8(255 - imagen.astype(np.int16))
```

Ajuste de brillo: desplaza todos los valores de intensidad sumando una constante c . Valores positivos aclaran la imagen y valores negativos la oscurecen. Se aplica saturación para mantener los valores dentro del rango válido $[0, 255]$:

$$I'(x, y) = \max(0, \min(255, I(x, y) + c))$$

$c > 0$ aclara; $c < 0$ oscurece

```
def ajuste_brillo(imagen, c):  
    return np.uint8(np.clip(imagen.astype(np.int16) + c, 0, 255))
```

Ajuste de contraste: escala los valores de intensidad multiplicando por un factor α . Un valor α mayor que 1 amplía la diferencia entre píxeles oscuros y claros, aumentando el contraste. La transformación lineal general combina ambos ajustes (Pratt, 2007, p. 243):

$$I'(x, y) = \alpha * I(x, y) + \beta$$

α : contraste, β : brillo

```
def ajuste_contraste(imagen, alpha, beta=0):
```

```
return np.uint8(np.clip(alpha * imagen.astype(np.float64) + beta, 0,
255))
```

Ecualización de histograma: técnica automática que redistribuye los niveles de gris para que el histograma de la imagen resultante sea aproximadamente uniforme, mejorando el contraste global sin necesidad de parámetros manuales (Gonzalez y Woods, 2018, pp. 177-182). Se basa en la función de distribución acumulada (CDF) del histograma normalizado:

Histograma normalizado:

$$p(k) = \frac{H(k)}{(M*N)}$$

$H(k)$ = número de píxeles con nivel k

CDF:

$$CDF(k) = \sum_{j=0}^{\{k\}} p(j)$$

suma acumulada de probabilidades

Codigo:

```
def ecualizacion_histograma(imagen):
    M, N = imagen.shape
    L = 256

    H = np.zeros(L)
    for k in range(L):
        H[k] = np.sum(imagen == k)
    p = H / (M * N) # p(k) = H(k) / (M*N)

    cdf = np.cumsum(p) # CDF(k) = Σ p(j) para j=0..k

    tabla_lut = np.round(cdf * (L - 1)).astype(np.uint8)
    imagen_ecualizada = tabla_lut[imagen]

    print("===Ecualizacion de Histograma ===")
    print(f" Rango original: [{imagen.min()}, {imagen.max()}]")
    print(f" Rango ecualizado: [{imagen_ecualizada.min()}, {imagen_ecualizada.max()}]")
    return imagen_ecualizada, p, cdf
```

4.2 Transformaciones de Nivel de Gris

Las transformaciones no lineales permiten mejorar zonas específicas de intensidad.

Transformación logarítmica: comprime las intensidades altas y expande las bajas, permitiendo visualizar detalles en zonas oscuras sin saturar las brillantes. Es especialmente útil para mostrar resultados de la transformada de Fourier:

$$I'(x, y) = c \cdot \log(1 + I(x, y))$$

c es una constante de escala para ajustar el rango

```
def transformacion_logaritmica(imagen, c=1.0):  
    resultado = c * np.log1p(imagen.astype(np.float64))  
    resultado = resultado / resultado.max() * 255  
    return np.uint8(resultado)
```

Corrección gamma: ajusta la luminosidad de la imagen con un comportamiento exponencial. Con valores de gamma menores que 1 se realzan las zonas oscuras; con valores mayores que 1 se realzan las zonas claras. Se utiliza para compensar la respuesta no lineal de monitores y sensores:

$$I'(x, y) = c \cdot I(x, y)^g$$

gamma < 1 aclara zonas oscuras; gamma > 1 oscurece

```
def correccion_gamma(imagen, gamma, c=1.0):  
    normalizada = imagen.astype(np.float64) / 255.0  
    corregida = c * np.power(normalizada, gamma) * 255.0  
    return np.uint8(np.clip(corregida, 0, 255))
```

Transformación por tramos (piecewise linear): define distintas funciones lineales para diferentes rangos de intensidad. Permite realzar selectivamente una banda de valores mientras atenúa las demás. Su aplicación conocida es el windowing en tomografía computarizada (TC), donde se ajusta el rango de densidades visualizadas para enfocarse en un tejido específico (hueso, músculo, pulmón).

```
def transformacion_tramos(imagen, puntos=[(0,0),(100,50),(200,230),(255,255)]):
    lut = np.zeros(256, dtype=np.uint8)
    for i in range(len(puntos) - 1):
        x0, y0 = puntos[i]
        x1, y1 = puntos[i + 1]
        for x in range(x0, x1 + 1):
            y = int(y0 + (y1 - y0) * (x - x0) / max(x1 - x0, 1))
            lut[x] = np.clip(y, 0, 255)
    return lut[imagen]
```

5. Filtrado y Convolución

5.1 Convolución Bidimensional

La convolución bidimensional es la base del filtrado espacial. El kernel se desliza sobre la imagen calculando una suma ponderada de los píxeles vecinos:

$$g(x,y) = \sum \sum f(u,v) \cdot h(x-u, y-v)$$

Código:

```
def aplicar_convolucion(imagen, kernel):
    filas, cols = imagen.shape
    kf, kc = kernel.shape
    pf, pc = kf // 2, kc // 2

    # Padding: anade bordes para no perder tamano
    img_pad = np.pad(imagen, ((pf, pf), (pc, pc)), mode='constant')
    img_filtrada = np.zeros_like(imagen, dtype=np.float64)

    # Bucle de convolucion - desliza el kernel
    for i in range(filas):
        for j in range(cols):
            region = img_pad[i:i+kf, j:j+kc].astype(np.float64)
            img_filtrada[i, j] = np.sum(region * kernel)

    return img_filtrada
```

5.2 Filtro de Media

Código:


```
def filtro_media(imagen, tam=3):
    kernel = np.ones((tam, tam), dtype=np.float64) / (tam * tam)
    resultado = aplicar_convolucion(imagen, kernel)
    return np.uint8(np.clip(resultado, 0, 255))
```

5.3 Filtro Gaussiano

Fórmula: $h(x,y) = (1/2\pi\sigma^2) \cdot \exp(-(x^2+y^2)/2\sigma^2)$

Código:

```
def filtro_gaussiano(imagen, sigma=1.0, tam=5):
    centro = tam // 2
    kernel = np.zeros((tam, tam), dtype=np.float64)
    for i in range(tam):
        for j in range(tam):
            x = i - centro
            y = j - centro
            # Formula gaussiana bidimensional - Pag. 28
            kernel[i, j] = (1 / (2 * np.pi * sigma**2)) * np.exp(-(x**2 + y**2) / (2 * sigma**2))
    kernel /= kernel.sum() # Normalizar para preservar brillo

    resultado = aplicar_convolucion(imagen, kernel)
    return np.uint8(np.clip(resultado, 0, 255))
```

5.4 Filtro de Mediana

Código:

```
def filtro_mediana(imagen, tam=3):
    filas, cols = imagen.shape
    pad = tam // 2
    img_pad = np.pad(imagen, pad, mode='constant')
    img_filtrada = np.zeros_like(imagen)

    for i in range(filas):
        for j in range(cols):
            vecindad = img_pad[i:i+tam, j:j+tam].flatten()
            vecindad_ord = np.sort(vecindad) # Paso 2: ordenar
            img_filtrada[i, j] = vecindad_ord[len(vecindad_ord) // 2] # Paso 3: mediana
    return img_filtrada
```

5.5 Filtro Laplaciano y LoG

Fórmula: $\nabla^2 f = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4 \cdot f(x,y)$

Código:

```
def filtro_laplaciano(imagen):
    kernel_laplaciano = np.array([
        [ 0, -1,  0],
        [-1,  4, -1],
        [ 0, -1,  0]
    ], dtype=np.float64)

    resultado = aplicar_convolucion(imagen, kernel_laplaciano)
    return np.uint8(np.clip(np.abs(resultado), 0, 255))

def filtro_log(imagen, sigma=1.0):
    suavizada = filtro_gaussiano(imagen, sigma=sigma)
    log = filtro_laplaciano(suavizada)
    return log
```

6. Detección de Bordos y Características

6.1 Gradiente, Magnitud y Orientación

Fórmulas:

- $\nabla f = [\partial f / \partial x, \partial f / \partial y]$
- $|\nabla f| = \sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2}$
- $\theta(x,y) = \arctan(\partial f / \partial y / \partial f / \partial x)$

Código:

python

```
def calcular_gradiente(imagen):
    img = imagen.astype(np.float64)
    # Derivadas parciales por diferencias finitas
    gx = np.zeros_like(img)
    gy = np.zeros_like(img)
    gx[:, 1:-1] = img[:, 2:] - img[:, :-2]    #  $\partial f / \partial x$ 
    gy[1:-1, :] = img[2:, :] - img[:-2, :]    #  $\partial f / \partial y$ 

    magnitud    = np.sqrt(gx**2 + gy**2)        #  $|\nabla f|$  exacta
    aprox_mag   = np.abs(gx) + np.abs(gy)       # Aproximacion eficiente
    orientacion = np.arctan2(gy, gx)            #  $\theta(x,y)$  en radianes

    return gx, gy, magnitud, aprox_mag, orientacion
```

6.2 Operador Sobel

Kernels:

- $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ → detecta bordes verticales
- $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$ → detecta bordes horizontales

Código:

```
def detectar_bordes_sobel(imagen):
    Gx_kernel = np.array([
        [-1, 0, 1],
        [-2, 0, 2],
        [-1, 0, 1]
    ], dtype=np.float64)

    Gy_kernel = np.array([
        [-1, -2, -1],
        [ 0,  0,  0],
        [ 1,  2,  1]
    ], dtype=np.float64)

    grad_x = aplicar_convolucion(imagen.astype(np.float64), Gx_kernel)
    grad_y = aplicar_convolucion(imagen.astype(np.float64), Gy_kernel)

    # Magnitud del gradiente
    magnitud = np.sqrt(grad_x**2 + grad_y**2)
    magnitud = np.clip(magnitud, 0, 255)
    return np.uint8(magnitud), grad_x, grad_y
```

6.3 Operador Prewitt

Código:

```
def detectar_bordes_prewitt(imagen):
    Gx_kernel = np.array([
        [-1, 0, 1],
        [-1, 0, 1],
        [-1, 0, 1]
    ], dtype=np.float64)

    Gy_kernel = np.array([
        [-1, -1, -1],
        [ 0,  0,  0],
        [ 1,  1,  1]
    ], dtype=np.float64)

    grad_x = aplicar_convolucion(imagen.astype(np.float64), Gx_kernel)
    grad_y = aplicar_convolucion(imagen.astype(np.float64), Gy_kernel)

    magnitud = np.sqrt(grad_x**2 + grad_y**2)
    return np.uint8(np.clip(magnitud, 0, 255))
)
```

6.4 Detector Canny

Pasos del algoritmo a mencionar:

1. Suavizado gaussiano
2. Cálculo de gradiente (magnitud y orientación)
3. Supresión no máxima → bordes de 1 píxel
4. Umbralización por histéresis (umbral alto + umbral bajo)

Código:

```
def detector_canny(imagen, sigma=1.0, umbral_bajo=50, umbral_alto=150):
    # Paso 1: Suavizado gaussiano
    suavizada = filtro_gaussiano(imagen, sigma=sigma)

    # Pasos 2-4 con OpenCV (implementacion optimizada)
    bordes = cv2.Canny(suavizada, umbral_bajo, umbral_alto)
    print(f"=== Detector Canny ===")
    print(f"  σ={sigma}, umbral_bajo={umbral_bajo}, umbral_alto={umbral_alto}")
    return bordes
```

7. Segmentación

El objetivo fundamental de la segmentación en este proyecto es separar los objetos de interés (los pupitres) del fondo del aula. Para lograr esto con precisión sin depender de funciones predefinidas de librerías como OpenCV, se implementó analíticamente desde cero el **Algoritmo Iterativo de Umbralización Global**.

1. Se inicializa una estimación para el umbral T calculando la intensidad media de la imagen.
2. Se particiona la imagen en dos conjuntos: el grupo $G1$, que contiene los píxeles correspondientes al pupitre ($f(x, y) > T$), y el grupo $G2$, que contiene los píxeles del fondo $f(x, y) \leq T$.
3. Se calculan las medias aritméticas m_1 y m_2 de ambos conjuntos y se actualiza el umbral ubicándolo en el punto medio exacto de ambas medias mediante la ecuación: $T = \frac{1}{2} (m_1 + m_2)$.

```
def umbralizacion_iterativa(imagen, tolerancia=1.0):

    T = np.mean(imagen)          # Inicializacion: media de la imagen
    T_viejo = -1
    iteraciones = 0

    print("=== Umbralizacion Iterativa ===")
    while abs(T - T_viejo) >= tolerancia:
        T_viejo = T

        G1 = imagen[imagen > T]   # Grupo objeto
        G2 = imagen[imagen <= T]  # Grupo fondo

        m1 = np.mean(G1) if len(G1) > 0 else 0
        m2 = np.mean(G2) if len(G2) > 0 else 0

        T = (m1 + m2) / 2.0        # T = (m1 + m2) / 2
        iteraciones += 1
        print(f" Iter {iteraciones:2d}: m1={m1:.2f}, m2={m2:.2f}, T={T:.4f}")

    imagen_binaria = (imagen > T).astype(np.uint8) * 255
    print(f" Umbral optimo encontrado: T* = {T:.4f} ({iteraciones} iteraciones)")
    return imagen_binaria, T
```

8. Morfología matemática

Al inicio de cada función, el algoritmo obtiene las dimensiones exactas de la imagen binaria y del elemento estructurante, posterior a esto se aplica un relleno con ceros en los bordes de la imagen mediante la función `np.pad`.

Para la erosión se calcula previamente una variable `suma_total` este valor nos sirve para que si el elemento estructurante se superpone sobre una región completamente blanca.

Mediante dos bucles anidados, el algoritmo recorre la imagen píxel por píxel, recortando en cada iteración una submatriz local (region) del mismo tamaño que el elemento estructurante. La restricción lógica se aplica multiplicando ambas matrices: si la suma del resultado es exactamente igual a la suma_total, significa que la matriz encajó por completo dentro del objeto. Solo en ese caso el píxel central sobrevive; de lo contrario, se convierte en fondo (0). Esto produce el efecto de adelgazamiento en los pupitres.

```
def erosion(imagen_binaria, elemento_estructurante):

    filas, cols = imagen_binaria.shape
    kf, kc = elemento_estructurante.shape
    pf, pc = kf // 2, kc // 2

    img_pad = np.pad(imagen_binaria, ((pf, pf), (pc, pc)), mode='constant')
    img_erosionada = np.zeros_like(imagen_binaria)
    suma_total = np.sum(elemento_estructurante) * 255

    for i in range(filas):
        for j in range(cols):
            region = img_pad[i:i+kf, j:j+kc]
            # Condicion: SE encaja completamente ( $\subseteq A$ )
            if np.sum(region * elemento_estructurante) == suma_total:
                img_erosionada[i, j] = 255
    return img_erosionada
```

La función de dilatación utiliza el mismo motor de recorrido matricial, pero modifica la condición de evaluación para lograr la expansión del objeto. Al multiplicar la submatriz local por el elemento estructurante, el algoritmo verifica únicamente que la suma sea mayor a cero > 0 .

Esta condición establece que basta con que un solo píxel de la matriz de 3×3 intersecte con un píxel blanco del objeto para que el píxel central sea marcado como verdadero (255). Esta lógica permite engrosar los contornos y es la base matemática que, combinada con la erosión, permite rellenar huecos y grietas del objeto

```
def dilatacion(imagen_binaria, elemento_estructurante):
    filas, cols = imagen_binaria.shape
    kf, kc = elemento_estructurante.shape
    pf, pc = kf // 2, kc // 2

    img_pad = np.pad(imagen_binaria, ((pf, pf), (pc, pc)), mode='constant')
    img_dilatada = np.zeros_like(imagen_binaria)

    for i in range(filas):
        for j in range(cols):
            region = img_pad[i:i+kf, j:j+kc]
            # Condicion: interseccion no vacia ( $n \cap A \neq \emptyset$ )
            if np.sum(region * elemento_estructurante) > 0:
                img_dilatada[i, j] = 255
    return img_dilatada
```

Una vez establecidas las iteraciones matriciales para la erosión y dilatación, el algoritmo implementa operaciones morfológicas mediante el anidamiento secuencial de estas funciones.

El código ejecuta primero una erosión sobre la imagen binaria original y pasa este resultado intermedio como argumento de entrada a la función de dilatación. Computacionalmente, este orden específico garantiza que los píxeles aislados o el ruido de fondo se eliminen por completo durante la erosión inicial.

```
def apertura_morfologica(imagen_binaria, SE=None):
    if SE is None:
        SE = np.ones((3, 3), np.uint8)
    erosionada = erosion(imagen_binaria, SE)
    return dilatacion(erosionada, SE)
```

Esta funcion lo que primero realiza es la dilatacion para primero expandir las matrices de los objetos y asi las pequeñas grietas o discontinuidades internas se fusionen al entrar en contacto. Para que luego la erosion lo regrese a su tamaño original pero ya los las grietas y discontinuidades corregidas.

```
def cierre_morfologico(imagen_binaria, SE=None):
    if SE is None:
        SE = np.ones((3, 3), np.uint8)
    dilatada = dilatacion(imagen_binaria, SE)
    return erosion(dilatada, SE)
```

Esta funcion lo que hace es obtener el contorno del objeto, realizando ambas operaciones de dilatacion y erosion y realizando la resta de ambas para unicamente quedarse con los bordes de color blanco

```
def gradiente_morfologico(imagen_binaria, SE=None):
    if SE is None:
        SE = np.ones((3, 3), np.uint8)
    dil = dilatacion(imagen_binaria, SE).astype(np.int16)
    ero = erosion(imagen_binaria, SE).astype(np.int16)
    return np.uint8(np.clip(dil - ero, 0, 255))
```

9. Extracción de características

En esta etapa, la imagen ya ha pasado por el preprocesamiento, la detección de bordes, la segmentación y las operaciones morfológicas. Como resultado obtenemos `img_cierre`, que representa una región binaria más limpia del objeto detectado. A partir de esta región vamos a extraer valores numéricos que permitan describir su forma y facilitar posteriormente su reconocimiento.

```
caract = extraer_caracteristicas_geometricas(img_cierre)
momentos_hu = calcular_momentos_hu(img_cierre)
H = calcular_histograma(img_gris)
_, desc_tex = calcular_glcmm(img_gris, d=1, theta=0)
visualizar_histograma(img_gris, f"Histograma - {nombre}")
```

Características geométricas

Área: La primera característica calculada es el área. En una imagen binaria, el área corresponde a la cantidad de píxeles que pertenecen al objeto. En nuestro código, los píxeles del objeto tienen valor 255, por lo que se cuentan únicamente esos píxeles.

```
def calcular_area(region_binaria):
    return int(np.sum(region_binaria == 255))
```

Perímetro: El perímetro se obtiene identificando la frontera del objeto. Para ello se aplica una erosión con un elemento estructurante en forma de cruz. La erosión elimina los píxeles externos; luego, al restar la imagen erosionada de la original, quedan solamente los píxeles del borde.


```
def calcular_perimetro(region_binaria):
    SE_cruz = np.array([[0,1,0],
                        [1,1,1],
                        [0,1,0]], dtype=np.uint8)

    interior = erosion(region_binaria, SE_cruz)
    frontera = region_binaria.astype(np.int16) - interior.astype(np.int16)
    frontera = np.clip(frontera, 0, 255).astype(np.uint8)

    return int(np.sum(frontera == 255))
```

Circularidad: y por último, con el área y el perímetro se calcula la circularidad

```
def calcular_circularidad(area, perimetro):
    if perimetro == 0:
        return 0.0
    return (4 * np.pi * area) / (perimetro ** 2)
```

Con esto podemos indicar qué tan parecido es el objeto a un círculo.

Momentos de Hu

Además de las características geométricas simples, implementamos los momentos de Hu. Estos son siete valores que describen la distribución espacial de la forma y permiten compararla aunque el objeto cambie de posición, tamaño o rotación.

Primero calculamos los momentos geométricos. Aquí cada píxel perteneciente al objeto se interpreta como 1, aunque la imagen binaria almacene valores 255. Esto evita que la intensidad altere el cálculo del área matemática del objeto

```
def calcular_momento_geometrico(imagen, p, q):
    filas, cols = imagen.shape
    momento = 0.0

    for y in range(filas):
        for x in range(cols):
            valor = 1.0 if imagen[y, x] > 0 else 0.0
            momento += (x ** p) * (y ** q) * valor

    return momento
```

A partir de los momentos m_{00} , m_{10} y m_{01} , calculamos el centroide del objeto:

```
centro_x = m10 / m00  
centro_y = m01 / m00
```

Después se calculan los momentos centrales, restando el centroide a cada coordenada. Esto elimina la dependencia de la posición del objeto en la imagen.

```
def calcular_momento_central(imagen, p, q, centro_x, centro_y):  
    filas, cols = imagen.shape  
    momento = 0.0  
  
    for y in range(filas):  
        for x in range(cols):  
            valor = 1.0 if imagen[y, x] > 0 else 0.0  
  
            dx = x - centro_x  
            dy = y - centro_y  
  
            momento += (dx ** p) * (dy ** q) * valor  
  
    return momento
```

Luego estos momentos se normalizan respecto al tamaño del objeto

Finalmente, se combinan los momentos normalizados para obtener los siete momentos de Hu.

```

hu[0] = eta20 + eta02

hu[1] = ((eta20 - eta02) ** 2) + (4 * (eta11 ** 2))

hu[2] = ((eta30 - 3 * eta12) ** 2) + ((3 * eta21 - eta03) ** 2)

hu[3] = ((eta30 + eta12) ** 2) + ((eta21 + eta03) ** 2)

hu[4] = (
    (eta30 - 3 * eta12)
    * (eta30 + eta12)
    * (((eta30 + eta12) ** 2) - 3 * ((eta21 + eta03) ** 2))
    +
    (3 * eta21 - eta03)
    * (eta21 + eta03)
    * (3 * ((eta30 + eta12) ** 2) - ((eta21 + eta03) ** 2))
)

hu[5] = (
    (eta20 - eta02)
    * (((eta30 + eta12) ** 2) - ((eta21 + eta03) ** 2))
    +
    4
    * eta11
    * (eta30 + eta12)
    * (eta21 + eta03)
)

hu[6] = (
    (3 * eta21 - eta03)
    * (eta30 + eta12)
    * (((eta30 + eta12) ** 2) - 3 * ((eta21 + eta03) ** 2))
    -
    (eta30 - 3 * eta12)
    * (eta21 + eta03)
    * (3 * ((eta30 + eta12) ** 2) - ((eta21 + eta03) ** 2))
)

```

Los siete valores obtenidos forman una firma numérica de la figura. Dos objetos con formas similares deberían producir momentos de Hu parecidos, incluso si uno aparece desplazado, escalado o rotado respecto al otro.

Histogramas

También calculamos el histograma normalizado de la imagen en escala de grises. Este indica qué proporción de píxeles posee cada nivel de intensidad entre 0 y 255.

```
def calcular_histograma(imagen, normalizar=True):
    M, N = imagen.shape
    L = 256
    H = np.zeros(L)
    for k in range(L):
        H[k] = np.sum(imagen == k)
    if normalizar:
        H = H / (M * N)
    return H
```

Por último, la textura se representa mediante una matriz GLCM. En nuestra implementación se comparan píxeles vecinos separados por una distancia de 1 en dirección horizontal. A partir de esta matriz se calculan tres descriptores: contraste, energía y entropía.

```

def calcular_glcmm(imagen, d=1, theta=0):
    L = 256
    glcm = np.zeros((L, L))
    filas, cols = imagen.shape

    if theta == 0:
        dx, dy = 0, d
    elif theta == 45:
        dx, dy = -d, d
    elif theta == 90:
        dx, dy = -d, 0
    elif theta == 135:
        dx, dy = -d, -d
    else:
        dx, dy = 0, d

    for i in range(filas):
        for j in range(cols):
            ni = i + dx
            nj = j + dy
            if 0 <= ni < filas and 0 <= nj < cols:
                glcm[imagen[i, j], imagen[ni, nj]] += 1
                glcm[imagen[ni, nj], imagen[i, j]] += 1

    total = glcm.sum()
    if total > 0:
        glcm /= total

    niveles = np.arange(L)
    i_idx, j_idx = np.meshgrid(niveles, niveles, indexing='ij')

    contraste = np.sum((i_idx - j_idx)**2 * glcm)
    energia = np.sum(glcm**2)
    entropia = -np.sum(glcm * np.log2(glcm + 1e-10))

```

El contraste mide la variación entre píxeles vecinos; la energía indica qué tan homogénea es la textura; y la entropía representa su grado de irregularidad.

10. Reconocimiento e interpretación

En esta etapa se implementaron tres procesos: la clasificación de escenas, la localización de un letrero específico y la segmentación de sus caracteres como paso previo al reconocimiento de texto.

En la ejecución principal se utilizan doce fotografías clasificadas en tres categorías: aula, mural y puesto de información.

```
etiquetas = (  
    ["Aula"] * 5  
    + ["Mural"] * 4  
    + ["Puesto de informacion"] * 3  
)
```

Clasificación de escenas mediante histogramas, HOG y KNN

Para reconocer a qué tipo de escena pertenece una fotografía, primero se construye un descriptor numérico. Este descriptor combina información de color, iluminación y estructura visual.

```
def descriptor_escena_desde_cero(imagen_rgb):  
  
    # Imagen pequena para acelerar el calculo del descriptor  
    imagen_pequena = redimensionar_vecino_mas_cercano(imagen_rgb, 160, 120)  
  
    gris = convertir_gris_reconocimiento(imagen_pequena)  
  
    hist_color = histograma_color_reducido(imagen_pequena, bins=8)  
    hist_gris = histograma_gris_reducido(gris, bins=16)  
  
    hog, _ = calcular_hog_desde_cero(  
        gris,  
        celdas_y=4,  
        celdas_x=4,  
        bins=9  
    )  
  
    # Cada grupo se normaliza para que HOG no domine  
    # solo por tener mas componentes.  
    hist_color = normalizar_l2(hist_color)  
    hist_gris = normalizar_l2(hist_gris)  
    hog = normalizar_l2(hog)  
  
    # HOG recibe algo mas de peso porque describe estructura.  
    descriptor = np.concatenate([  
        hist_color,  
        hist_gris,  
        2.0 * hog  
    ])  
  
    return descriptor
```

El histograma de color divide cada canal RGB en ocho intervalos, produciendo veinticuatro valores que resumen los colores predominantes. El histograma de grises utiliza dieciséis intervalos para representar la distribución de luminosidad.

El componente principal es HOG, o Histograma de Gradientes Orientados. Este algoritmo calcula los cambios de intensidad de la imagen y obtiene la orientación de sus bordes. Después divide la imagen en una cuadrícula de 4×4 celdas y, dentro de cada una, acumula las direcciones de borde en nueve intervalos angulares.

Así, HOG no se centra en píxeles aislados, sino en la estructura de la escena: líneas, bordes, esquinas y distribuciones de formas. En el descriptor final se le asigna mayor peso para poder distinguir mejor un aula, un mural o un puesto de información.

Una vez creado el descriptor, se aplica KNN. Este algoritmo compara la imagen evaluada con las demás mediante distancia euclidiana y asigna la clase de la fotografía más parecida.

```
for i in range(len(datos_entrenamiento)):

    diferencia = descriptor_prueba - datos_entrenamiento[i]

    distancia = np.sqrt(np.sum(diferencia ** 2))

    distancias.append(
        (distancia, etiquetas[i], nombres_entrenamiento[i])
    )

distancias.sort(key=lambda elemento: elemento[0])

vecinos = distancias[:K]

votos = {}

for _, etiqueta, _ in vecinos:
    if etiqueta not in votos:
        votos[etiqueta] = 0

    votos[etiqueta] += 1

clase_predicha = max(votos, key=votos.get)
```

La evaluación es de tipo *leave-one-out*: cada fotografía se prueba usando las once restantes como referencia. Finalmente, el programa muestra la clase predicha, los vecinos encontrados, la exactitud total y una matriz de confusión.

Localización del letrero mediante Template Matching con NCC

El segundo proceso busca localizar el letrero **INFORMACIONES**. Para ello se extrae una plantilla limpia desde Image10.jpeg, y posteriormente se intenta encontrar ese mismo letrero en otras imágenes.

```
plantilla_rgb = recortar_region_relativa(  
    imagen_plantilla,  
    0.245, 0.230,  
    0.675, 0.390  
)
```

La comparación se realiza mediante NCC, o Correlación Cruzada Normalizada. Este método compara la plantilla con cada región candidata de la imagen y produce un valor de similitud. Al restar la media de intensidades antes de comparar, el método es menos sensible a diferencias generales de brillo.

```
region_normalizada = valores_region - media_region  
plantilla_normalizada = valores_plantilla - media_plantilla  
  
denominador_region = np.sqrt(  
    np.sum(region_normalizada ** 2)  
)  
  
denominador_plantilla = np.sqrt(  
    np.sum(plantilla_normalizada ** 2)  
)  
  
denominador = denominador_region * denominador_plantilla  
  
if denominador == 0:  
    return -2.0, proporcion_visible  
  
numerador = np.sum(  
    region_normalizada * plantilla_normalizada  
)  
  
ncc = numerador / denominador
```


La búsqueda no utiliza una única escala. La plantilla se redimensiona manualmente mediante interpolación bilineal y se prueba con diferentes tamaños, porque el letrero puede aparecer más grande o más pequeño según la distancia de captura.

Para Image12.jpeg, donde el letrero está parcialmente cubierto por vegetación, se implementa una máscara basada en exceso de verde:

```
exceso_verde = 2 * G - R - B

es_vegetacion = (
    G >= verde_minimo
    and G >= R + diferencia_minima
    and G >= B + diferencia_minima
    and exceso_verde > umbral_exceso
)
```

Los píxeles detectados como hojas o ramas se ignoran durante la comparación NCC. De esta manera, la zona ocluida no penaliza directamente la coincidencia con la plantilla.

Segmentación del texto como preprocesamiento OCR

Finalmente, la última implementación que se realiza es el procesado de la palabra **INFORMACIONES** letra por letra del letrero. Primero se recorta la región del texto y se convierte a escala de grises.

```
region_texto = recortar_region_relativa(
    imagen10_original,
    0.258, 0.265,
    0.645, 0.345
)
```

Después se implementa el método de Otsu, que recorre los posibles umbrales y selecciona aquel que maximiza la separación entre fondo y texto.

```

varianza_entre_clases = (
    peso_fondo
    * peso_texto
    * ((media_fondo - media_texto) ** 2)
)

if varianza_entre_clases > mejor_varianza:
    mejor_varianza = varianza_entre_clases
    mejor_umbral = umbral

```

Como las letras son blancas sobre un fondo azul, se establece además un mínimo de intensidad para evitar confundir reflejos con caracteres.

```

umbral_final = max(umbral_otsu, 180)

binaria = np.zeros_like(gris, dtype=np.uint8)
binaria[gris > umbral_final] = 255

```

La línea de texto se localiza mediante una proyección horizontal, contando píxeles blancos por fila. Luego, sobre esa línea, una proyección vertical cuenta píxeles blancos por columna para separar cada carácter.

```

proyeccion_horizontal = np.sum(binaria == 255, axis=1)

lineas = encontrar_intervalos_activos(
    proyeccion_horizontal,
    minimo_actividad=max(2, int(region_texto.shape[1] * 0.03)),
    longitud_minima=3
)

proyeccion_vertical = np.sum(renglon == 255, axis=0)

caracteres = encontrar_intervalos_activos(
    proyeccion_vertical,
    minimo_actividad=max(1, int(renglon.shape[0] * 0.05)),
    longitud_minima=3
)

```

```

def encontrar_intervalos_activos(proyeccion,
                                minimo_actividad=1,
                                longitud_minima=2):

    intervalos = []

    inicio = None

    for i in range(len(proyeccion)):

        activo = proyeccion[i] >= minimo_actividad

        if activo and inicio is None:
            inicio = i

        elif not activo and inicio is not None:

            if i - inicio >= longitud_minima:
                intervalos.append((inicio, i - 1))

            inicio = None

    if inicio is not None:
        if len(proyeccion) - inicio >= longitud_minima:
            intervalos.append((inicio, len(proyeccion) - 1))

    return intervalos

```

Como resultado, el programa encierra visualmente cada letra segmentada y comprueba si la cantidad obtenida coincide con las trece letras de la palabra **INFORMACIONES**.